

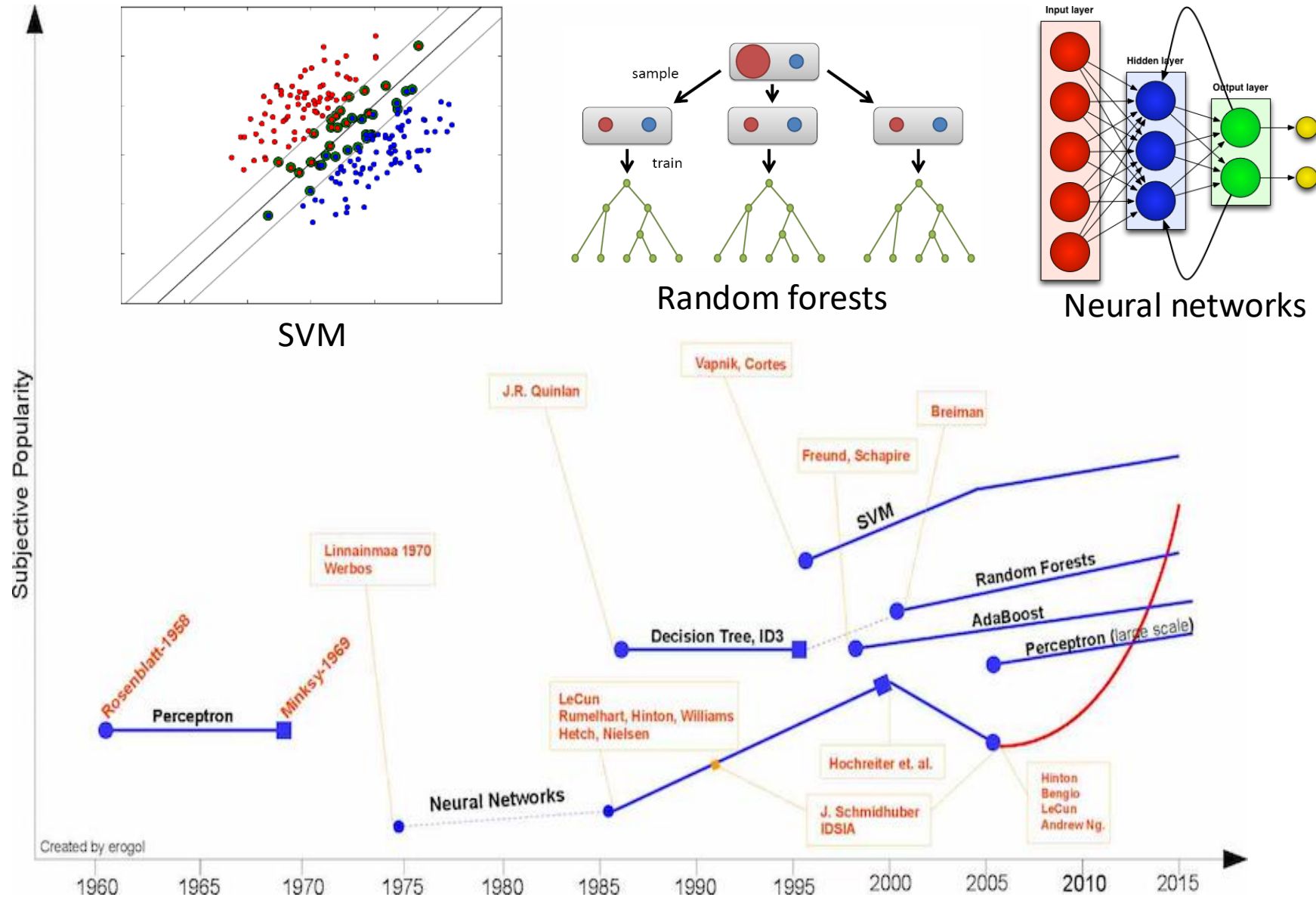
Convolutional Neural Networks

Yu Kong

Department of Computer Science and Engineering

Michigan State University

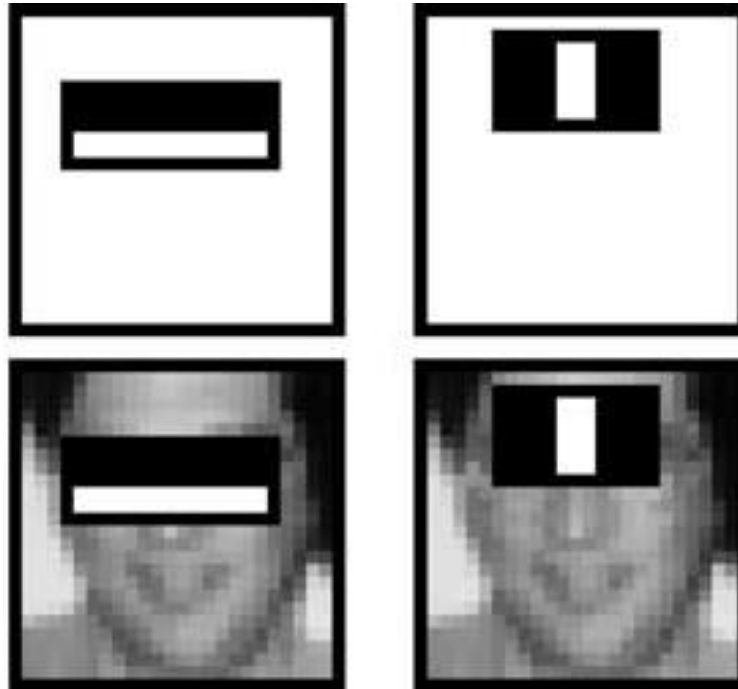
Introduction



Introduction

Low-level observation $\rightarrow$ mid-level features $\rightarrow$ high-level semantics

100	103	157	...
180	198	175	...
178	198	104	...
...			...



Deep learning

- Learn a *feature hierarchy* all the way from pixels to classifier
- Each layer extracts features from the output of previous layer
- **Train all layers jointly (backpropagation)**



Background

- A Convolutional Neural Network (CNN) consists of one or more **convolutional layers** (often with a **subsampling** step and an **activation** step) and then followed by one or more **fully connected layers** as observed in a standard multilayer neural network.

LeNet-5

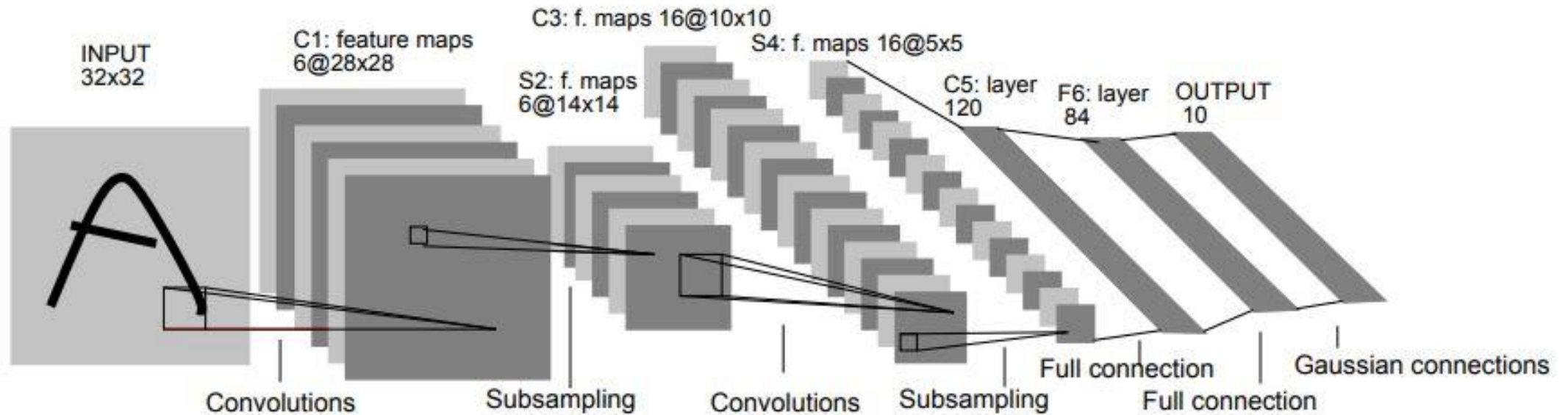
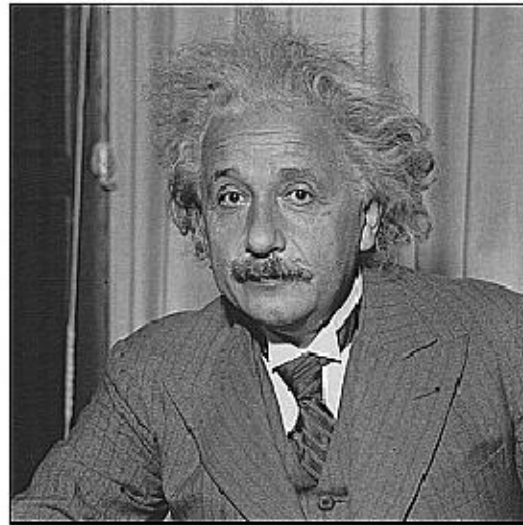
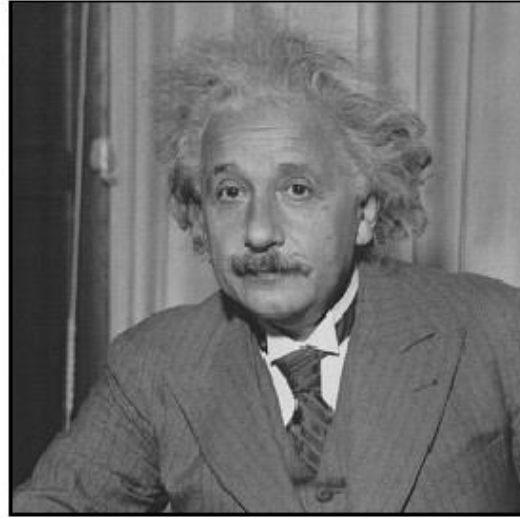


Image Filters



Smooth/Sharpen Images... Find edges...

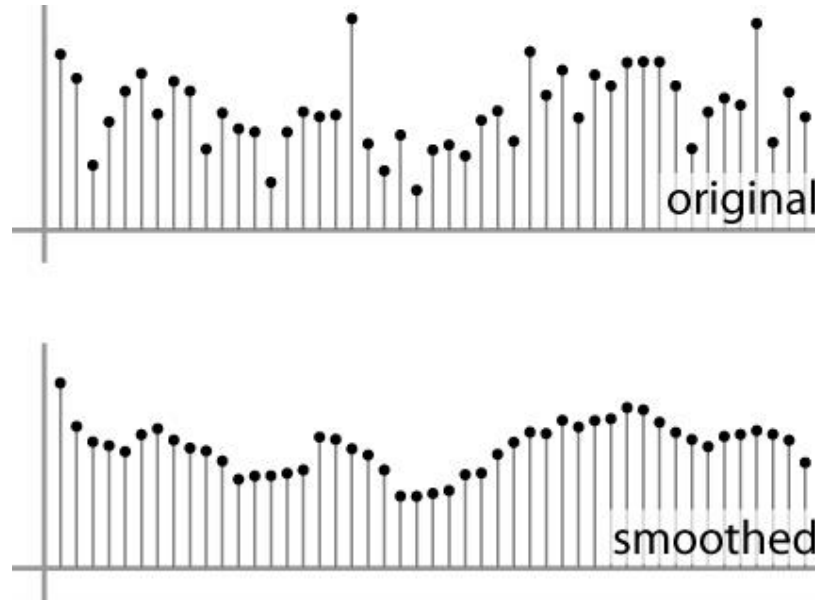
Images as functions

- We can think of an image as a function, f , from R^2 to R :
 - $f(x, y)$ gives the intensity at position (x, y)
 - Realistically, we expect the image only to be defined over a rectangle, with a finite range:
 - $f: [a, b] \times [c, d] \rightarrow [0, 1.0]$
- A color image is just three functions pasted together. We can write this as a “vector-valued” function:

$$f(x, y) = \begin{bmatrix} r(x, y) \\ g(x, y) \\ b(x, y) \end{bmatrix}$$

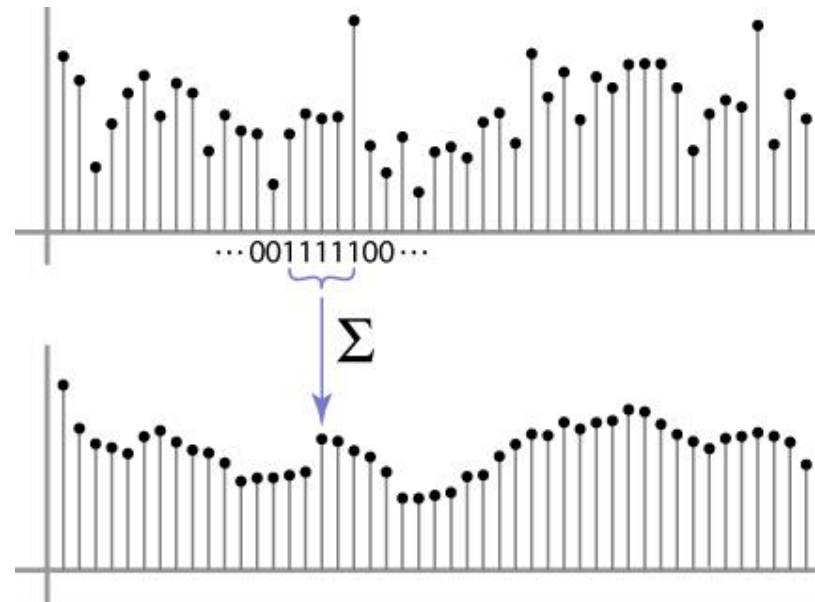
First attempt at a solution

- Let's replace each pixel with an average of all the values in its neighborhood
- Moving average in 1D:



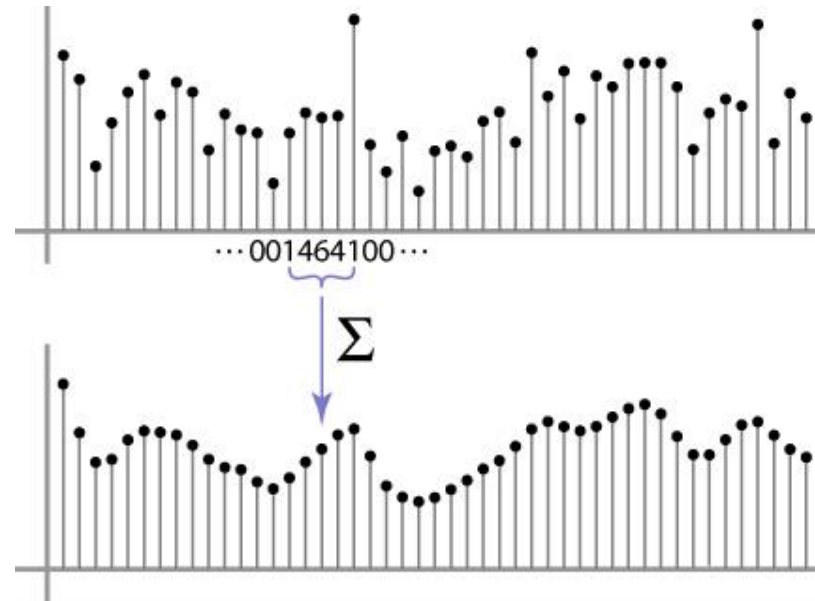
Weighted Moving Average

- Can add weights to our moving average
- *Weights* $[1, 1, 1, 1, 1] / 5$ (kernel)



Weighted Moving Average

- Non-uniform weights $[1, 4, 6, 4, 1] / 16$



Moving Average In $\frac{1}{9}$

$F[x, y]$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$G[x, y]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0								

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10							

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20						

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30					

Moving Average In 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30	30				

Moving Average In 2D

$$F[x, y]$$

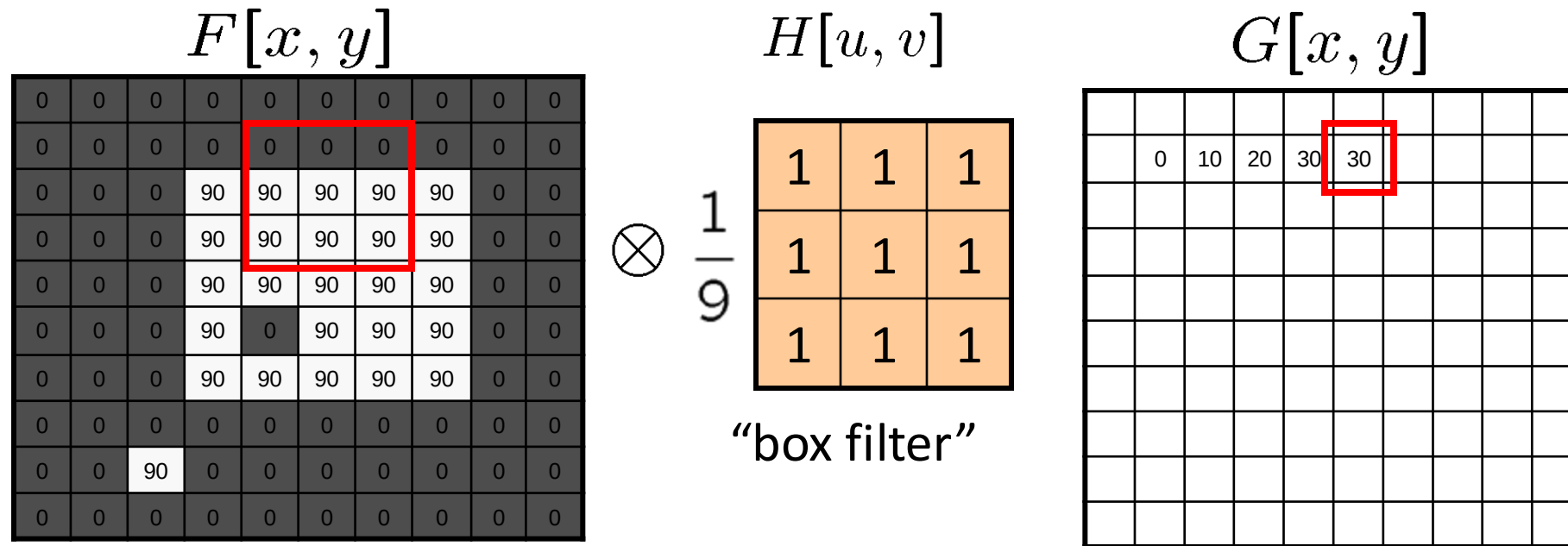
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Averaging filter

- What values belong in the kernel H for the moving average example?



$$G = H \otimes F$$

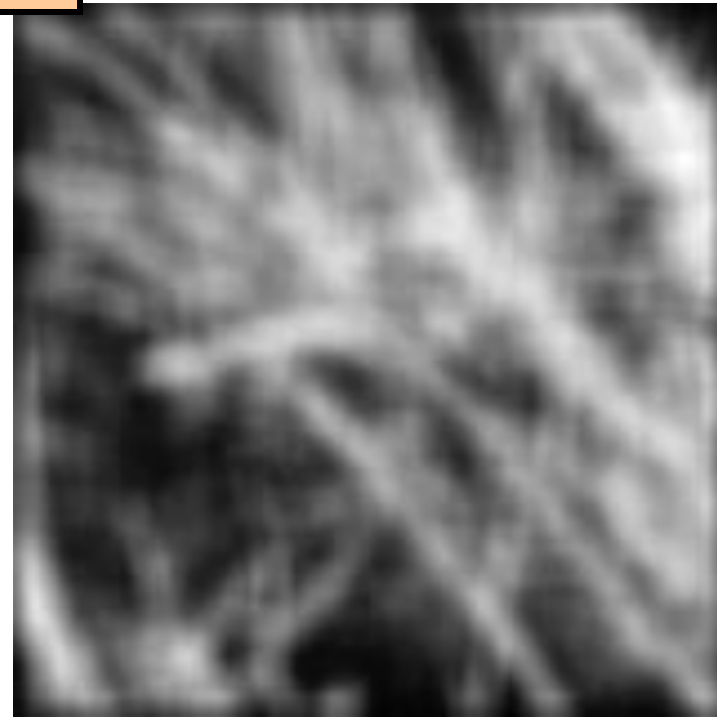
Smoothing by averaging

1	1	1
1	1	1
1	1	1

depicts box filter



original



filtered

Gaussian filter

- What if we want nearest neighboring pixels to have the most influence on the output?

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

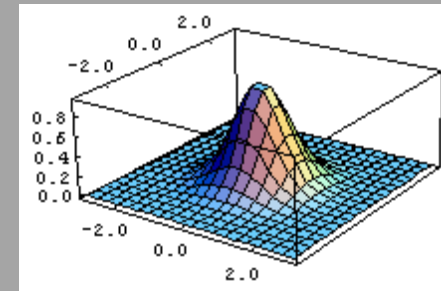
$F[x, y]$

$\frac{1}{16}$

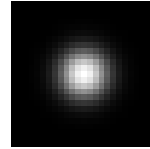
1	2	1
2	4	2
1	2	1

$H[u, v]$

This kernel is an approximation of a Gaussian function:

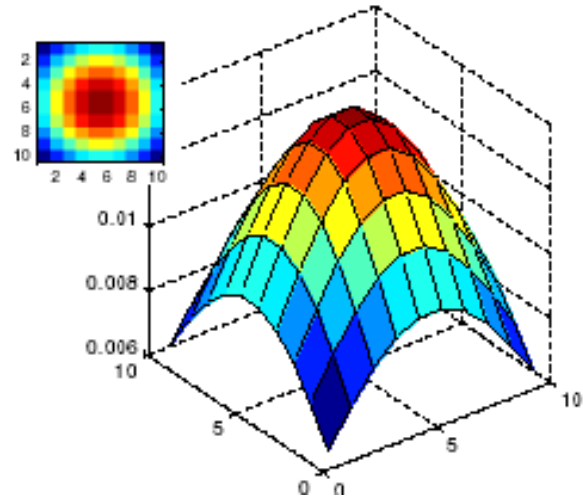


Smoothing with a Gaussian

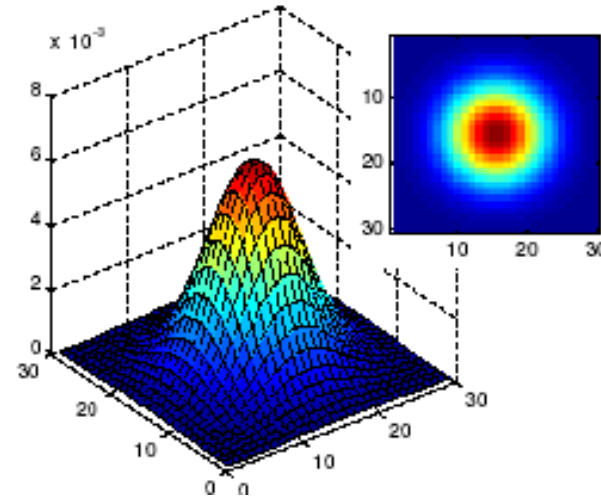


Gaussian filters

- What parameters matter here?
- **Size** of kernel or mask
 - Note, Gaussian function has infinite support, but discrete filters use finite kernels



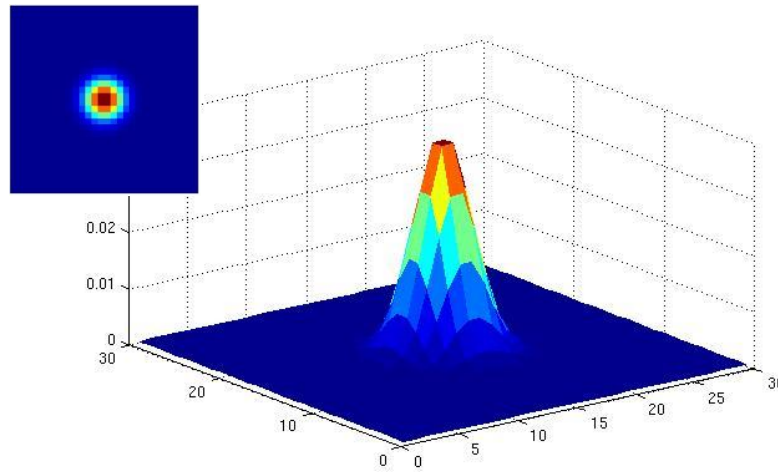
$\sigma = 5$ with
10 x 10
kernel



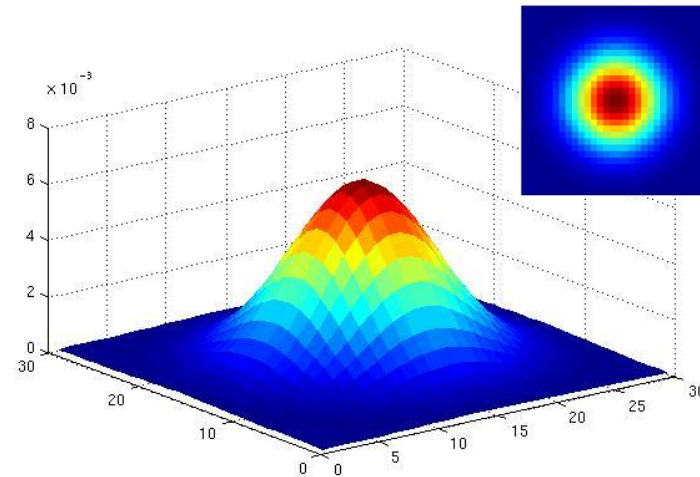
$\sigma = 5$ with
30 x 30
kernel

Gaussian filters

- What parameters matter here?
- **Variance** of Gaussian: determines extent of smoothing



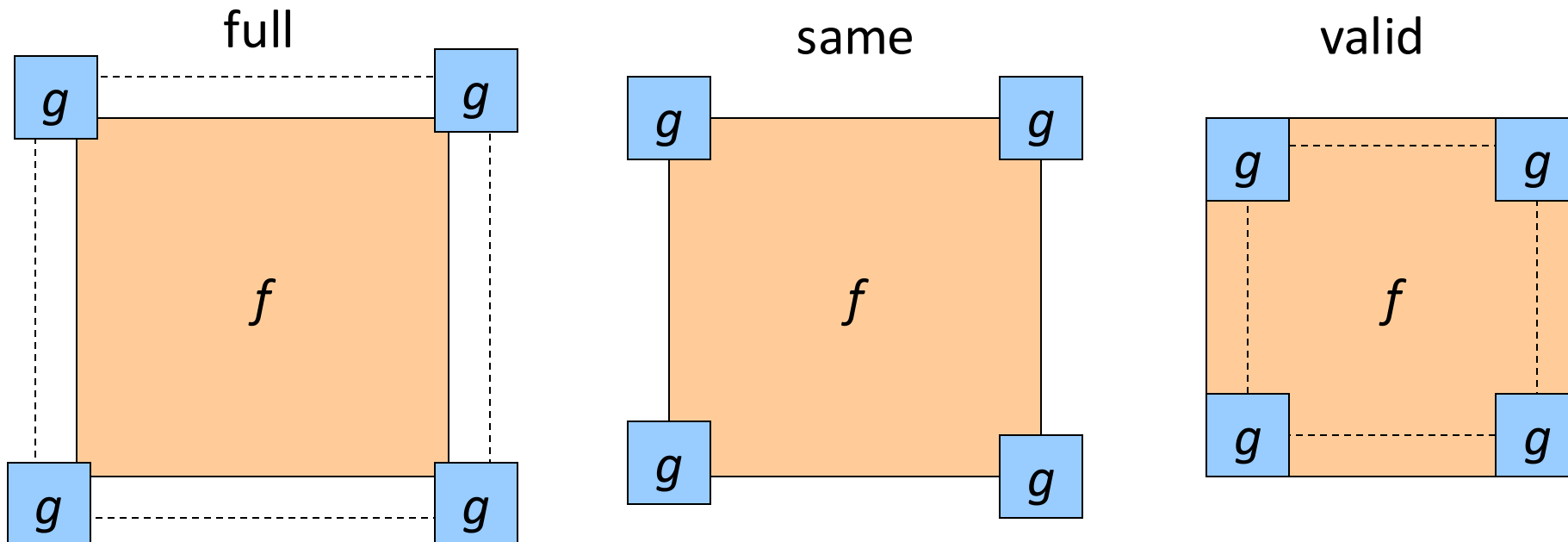
$\sigma = 2$ with
30 x 30
kernel



$\sigma = 5$ with
30 x 30
kernel

Boundary issues

- What is the size of the output?
- MATLAB: `filter2(g, f, shape)`
 - *shape* = 'full': output size is sum of sizes of *f* and *g*
 - *shape* = 'same': output size is same as *f*
 - *shape* = 'valid': output size is difference of sizes of *f* and *g*



Boundary issues

- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - methods (MATLAB):
 - clip filter (black): `imfilter(f, g, 0)`
 - wrap around: `imfilter(f, g, 'circular')`
 - copy edge: `imfilter(f, g, 'replicate')`
 - reflect across edge: `imfilter(f, g, 'symmetric')`

Filtering an impulse signal

What is the result of filtering the impulse signal (image) F with the arbitrary kernel H ?

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

$F[x, y]$



a	b	c
d	e	f
g	h	i

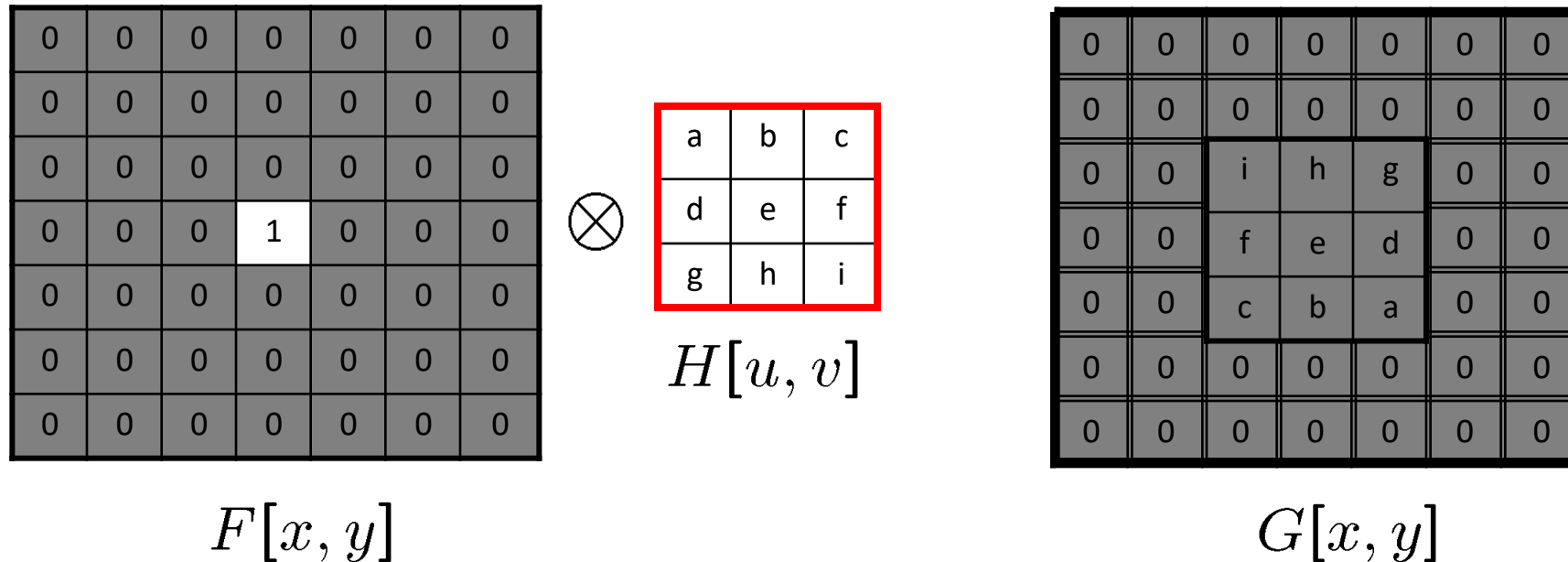
$H[u, v]$

$G[x, y]$

Filtering an impulse signal (correlation)

$$G = H \otimes F$$

What is the result of filtering the impulse signal (image) F with the arbitrary kernel H ?

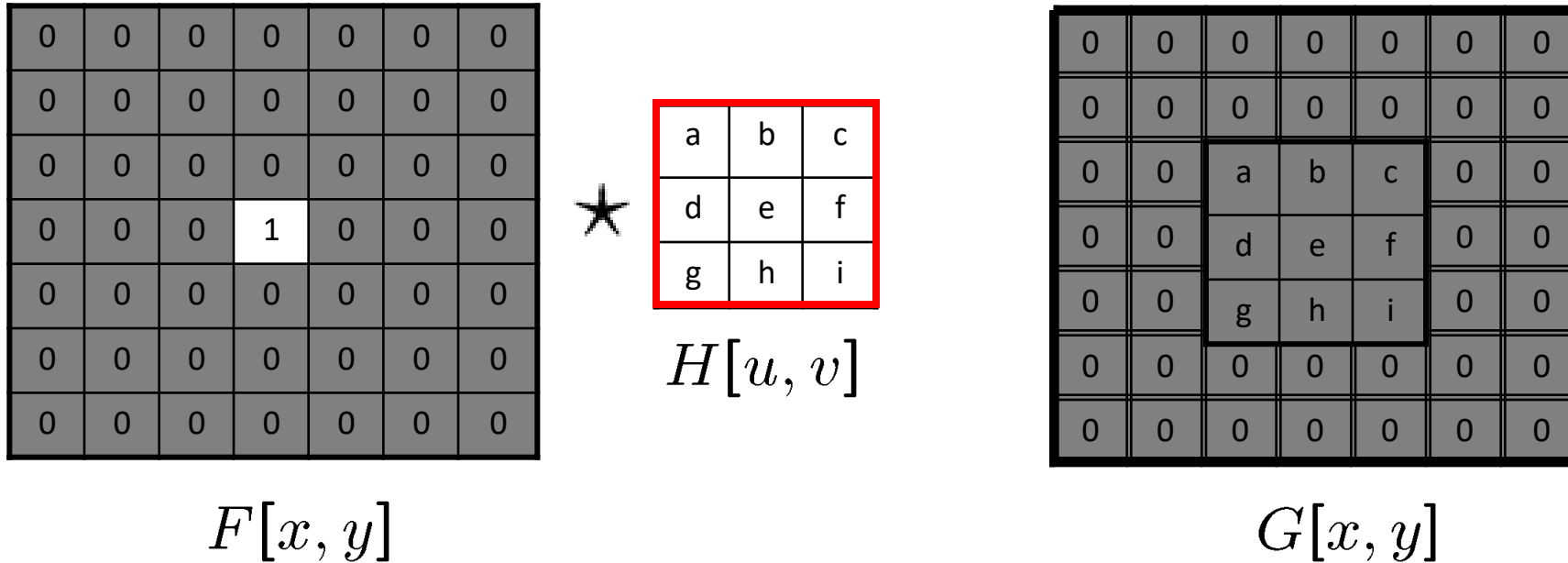


$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

Filtering an impulse signal (convolution)

$$G = H \star F$$

What is the result of filtering the impulse signal (image) F with the arbitrary kernel H ?



$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

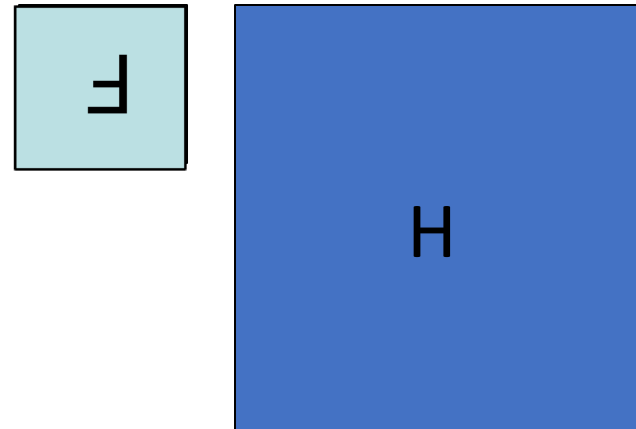
Convolution

- Convolution:
 - Flip the filter in both dimensions (bottom to top, right to left)
 - Then apply cross-correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H \star F$$

↑
*Notation for
convolution
operator*



- Image $A = \begin{bmatrix} 17 & 24 & 1 & 8 & 15 \\ 23 & 5 & 7 & 14 & 16 \\ 4 & 6 & 13 & 20 & 22 \\ 10 & 12 & 19 & 21 & 3 \\ 11 & 18 & 25 & 2 & 9 \end{bmatrix}$

- Kernel $h = \begin{bmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{bmatrix}$

- Output? $1 \times 2 + 8 \times 9 + 15 \times 4 + 7 \times 7 + 14 \times 5 + 16 \times 3 + 13 \times 6 + 20 \times 1 + 29 \times 8 = 575$

Convolving with linear filters



Original

0	0	0
0	0	1
0	0	0

?

Convolving with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted left
by 1 pixel with
correlation

Practice with linear filters



Original

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

Practice with linear filters



Original

 $\frac{1}{9}$

1	1	1
1	1	1
1	1	1



Blur (with a
box filter)